DISEÑO DE PATRONES – UTP – SEMESTRE I 2026 – SEMANA 2
Ejerc. - PRINCIPIO Abierto/Cerrado (Open/Closed Principle - OCP)

Ejercicio 1: Descuentos en productos
Se requiere calcular descuentos para distintos tipos de clientes. El código actual modifica la misma clase cada vez que aparece un nuevo tipo de cliente, rompiendo el principio abierto/cerrado.

CODIGO INCORRECTO

```
class CalculadoraDescuento {
    public double calcular(String tipoCliente, double precio) {
        if (tipoCliente.equals("VIP")) {
            return precio * 0.8;
        } else if (tipoCliente.equals("Regular")) {
            return precio * 0.9;
        } else {
            return precio;
        }
    }
}
```

CODIGO CORRECTO (ABIERTO CERRADO)

```
interface EstrategiaDescuento {
    double aplicar(double precio);
}

class DescuentoVIP implements EstrategiaDescuento {
    public double aplicar(double precio) { return precio * 0.8; }
}

class DescuentoRegular implements EstrategiaDescuento {
    public double aplicar(double precio) { return precio * 0.9; }
}

class SinDescuento implements EstrategiaDescuento {
    public double aplicar(double precio) { return precio; }
}

class CalculadoraDescuento {
    public double calcular(EstrategiaDescuento estrategia, double precio) {
        return estrategia.aplicar(precio);
```

```
    }
}
```

Ejercicio 2: Reportes
Se necesita generar reportes en distintos formatos (PDF, Excel, etc.). El código actual modifica la clase Reporte cada vez que se agrega un nuevo formato.

CODIGO INCORRECTO

```
class Reporte {
    public void generar(String tipo) {
        if (tipo.equals("PDF")) {
            System.out.println("Generando reporte en PDF");
        } else if (tipo.equals("Excel")) {
            System.out.println("Generando reporte en Excel");
        }
    }
}
```

CODIGO CORRECTO (ABIERTO/CERRADO)

```
interface GeneradorReporte {
    void generar();
}

class ReportePDF implements GeneradorReporte {
    public void generar() {
        System.out.println("Generando reporte en PDF");
    }
}

class ReporteExcel implements GeneradorReporte {
    public void generar() {
        System.out.println("Generando reporte en Excel");
    }
}

class ReporteService {
    public void generar(GeneradorReporte generador) {
        generador.generar();
    }
```

```
}
```

Ejercicio 3: Métodos de pago
Se requiere procesar pagos con distintos métodos (tarjeta, PayPal, etc.). El código actual modifica la clase ProcesadorPago cada vez que se agrega un nuevo método.

CODIGO INCORRECTO

```
class ProcesadorPago {
    public void procesar(String metodo) {
        if (metodo.equals("Tarjeta")) {
            System.out.println("Procesando pago con tarjeta");
        } else if (metodo.equals("PayPal")) {
            System.out.println("Procesando pago con PayPal");
        }
    }
}
```

CODIGO CORRECTO (ABIERTO/CERRADO)

```
interface MetodoPago {
    void procesar();
}

class PagoTarjeta implements MetodoPago {
    public void procesar() {
        System.out.println("Procesando pago con tarjeta");
    }
}
class PagoPayPal implements MetodoPago {
    public void procesar() {
        System.out.println("Procesando pago con PayPal");
    }
}

class ProcesadorPago {
    public void procesar(MetodoPago metodo) {
        metodo.procesar();
    }
}
```